实验报告

一、源码

```
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

int main()
{
	ofstream ofile;//定义输出流对象
	ifstream ifile;//定义输入流对象
	ofile.open("first.dat",ios::out | ios::binary);//以二进制的方式打开文件 first.dat

	//从键盘输入 5 个整数到 first.dat
	if (ofile)
	{
		for (int i=0;i<5;i++)
		{
			int temp;
			cin >> temp;
			ofile.write(reinterpret_cast<char*>(&temp),sizeof(int));//将 temp 中 5 个数写入文件输出流中，长度为 int 字节数
		}
		ofile.close();//因为后续要再同一流对象上打开另外的文件，因此需要使用 close（）函数关闭当前文件

		//输出当前 first.dat 中的数字
		cout << "first.dat:" << endl;
		ifile.open("first.dat",ios::in | ios::binary);
		while (ifile.peek() != EOF)//peek（）为取文件当前指针位置，EOF 为文件尾标识符
		{
			int temp;
			ifile.read(reinterpret_cast<char *>(&temp),sizeof(int));
			cout << temp <<" ";
		}
		ifile.close();
		cout << endl;
	}
	else
	{
		cout << "ERROR:Failed to open the file ""first.dat""." << endl;
```

```
}

//从键盘输入 5 个整数到 second.dat
ofile.open("second.dat",ios::out | ios::binary);
if (ofile)//判定文件是否被打开，不是返回 0
{
	for (int i=0;i<5;i++)
	{
		int temp;
		cin >> temp;
		ofile.write(reinterpret_cast<char*>(&temp),sizeof(int));
	}
	ofile.close();

	//输出当前 second.dat 中的数字
	cout << "second.dat:" << endl;
	ifile.open("second.dat",ios::in | ios::binary);
	while (ifile.peek() != EOF)
	{
		int temp;
		ifile.read(reinterpret_cast<char *>(&temp),sizeof(int));
		cout << temp <<" ";
	}
	ifile.close();
	cout << endl;
}
else
{
	cout << "ERROR:Failed to open the file ""second.dat""." << endl;
}

	//从 second.dat 中读取 5 个整数，存放到 first.dat 原有数据的后面
ofile.open("first.dat",ios::app | ios::binary);//打开一个输出文件 first.dat，并仅用于在文件尾添加数据，不会删除原先内容
ifile.open("second.dat",ios::in | ios::binary);//打开一个输入文件 second.dat

if (ofile && ifile)//判断两个文件是否被成功打开
{
	while(ifile.peek() != EOF)
	{
		char temp;
		ifile.read(reinterpret_cast<char *>(&temp),sizeof(char));//从 first.dat 中读入数据
		ofile.write(reinterpret_cast<char *>(&temp),sizeof(char));//写入 second.dat 中
	}
```

```
        ifile.close();
        ofile.close();

        //输出当前 first.dat 中的数字
        cout << "first.dat:" << endl;
        ifile.open("first.dat",ios::in | ios::binary);
        while (ifile.peek()!=EOF)
        {
            int temp;
            ifile.read(reinterpret_cast<char *>(&temp),sizeof(int));
            cout << temp << " ";
        }
        ifile.close();
        cout << endl;
    }

    //从 first.dat 读取 10 个整数，升序排序后存放到 second.dat
    int arr[11]={0};
    int i=0;
    ifile.open("first.dat",ios::in | ios::binary);
    ofile.open("second.dat",ios::out  |  ios::binary);//由于需要覆盖原来 second.dat 中原先的数字，因此可直接使用 out，
    //隐含 ios::trunc（若文件已存在则删除其中原先内容）

    if (ofile && ifile)
    {

        while (ifile.peek() != EOF)
        {
            int temp;
            ifile.read(reinterpret_cast<char *>(&temp),sizeof(int));
            arr[i++]=temp;//将从 frist.dat 中读取出来的数据存入数组中
        }
        ifile.close();

        sort(arr,arr+10);//通过 sort（）对数组进行升序排序

        for (int j=0;j<i;j++)
        {
            ofile.write(reinterpret_cast<char  *>(&arr[j]),sizeof(int));//将升序后数组中的数字输入 second.dat 中
        }
        ofile.close();
```

```
        //输出当前 second.dat 中的数字
        cout << "second.dat:" << endl;
        ifile.open("second.dat",ios::in | ios::binary);
        while (ifile.peek() != EOF)
        {
            int temp;
            ifile.read(reinterpret_cast<char *>(&temp),sizeof(int));
            cout << temp << " ";
        }
        ifile.close();
        cout << endl;
    }

    return 0;
}
```

二、反思

1、在一开始，由于想要同时熟悉一下文本模式和二进制模式两种不同的输入输出写法，再向文件写入数据是采用了文本模式，在后续处理文件中已有数据的时候采用了二进制模式获取以及处理，倒是最终输出时由于格式不统一，出现混乱以及莫名其妙的错误。一般写的时候还是统一用 read/write（binary）的方式写，或者就 << 和 >> 配对使用，二者不要混起来用。

2、在判断文件是否读到文件尾时，一开始所使用的是 while (!ifile.eof())，会发现文件中只有五个数字，但最终输出时却输出了 6 次，最后一位重复了两次。原因是 eof（）返回 true 的条件是“读到文件结束符”，而“文件结束符”并不是文件最后一个内容，而是最后一个字符下一个才是。

```
bool eof()
{
    if (eofbit)
```

```
        return true;
    else
        return false;
}
```

在实现过程中，eofbit 的延迟更新就会导致多一次循环，但因为没读进去东西，因此输出缓冲区的最后一个读入。而改为 while (ifile.peek() != EOF)之后，则相当于手动判断是否遇到“文件结束符”，不会存在延迟更新的情况。